

Vehicle Traffic Counter

A Computer Vision Project — Detection, Tracking & Counting

Flipped Course Evaluation — Build Your Own Project

Course Domain: Computer Vision

Submitted as: GitHub Repository + Project Report (PDF)

1. Introduction

This project implements a command-line computer vision system that detects, tracks, and counts vehicles in traffic video footage. It combines a pretrained deep-learning object detector (YOLOv8n) with a custom-built tracking algorithm to solve a problem detection alone cannot: knowing that the car seen in frame 40 is the same car seen in frame 55, so it is counted once rather than once per frame it appears in.

The system is designed as three independent, testable modules — detection, tracking/counting, and reporting — wired together by a single CLI entry point. It requires no GUI and no manual dataset labeling; the pretrained model handles recognition, while the original contribution of this project is the identity-tracking and line-crossing counting logic, plus the reporting pipeline built around it.

2. Problem Statement

Manual vehicle counting for traffic analysis is slow, error-prone, and does not scale to long durations of footage. Traffic authorities, researchers, and students need an automated, repeatable way to count vehicles passing a point in a video, broken down by vehicle type, without expensive dedicated hardware or a full traffic-analytics platform.

Scope

The project counts vehicles from a fixed-camera or roughly stable viewpoint (CCTV, dashcam, drone hover footage). It deliberately excludes multi-camera fusion, vehicle speed estimation, and license-plate recognition, keeping the system focused and achievable.

Target Users

- Traffic/transport researchers needing a lightweight, scriptable vehicle counting tool
- Students learning how to combine a pretrained detector with a custom tracking algorithm
- Anyone wanting quick, offline-after-setup vehicle counts from a video clip

3. Functional Requirements

- **FR1 — Detection:** Detect vehicles (car, motorcycle, bus, truck) in each video frame using a pretrained YOLOv8n model, filtering out all non-vehicle classes.
- **FR2 — Tracking & Counting:** Assign a persistent ID to each vehicle across frames using a custom centroid-tracking algorithm, and count each unique vehicle exactly once when it crosses a configurable virtual line.
- **FR3 — Reporting:** Produce an annotated output video (boxes, IDs, running counts), a per-frame CSV log, and a final JSON summary broken down by vehicle class.

- **FR4 — CLI configurability:** Allow the user to configure the input video, output directory, counting-line position, detection confidence threshold, model path, and frame-skip rate via command-line flags.

Input / Output Structure

Input: a single video file path (any format OpenCV can decode, e.g. .mp4, .avi) plus optional CLI flags.

Output: an annotated video file, a CSV file logging counts over time, and a JSON file with final totals — all written to a user-specified output directory.

User Workflow

1) User runs `python src/main.py --input video.mp4` from the terminal → 2) the tool validates the input and loads the model → 3) it processes the video frame-by-frame, printing progress → 4) on completion it prints a summary to the terminal and writes the three report files.

4. Non-Functional Requirements

Requirement	How it is addressed
Performance	The <code>--skip-frames</code> flag lets detection run on every Nth frame, trading accuracy for throughput on CPU-only machines.
Reliability	Input path, video-open success, and CLI argument ranges (e.g. <code>--line-position</code> , <code>--confidence</code>) are validated upon startup.
Usability	Every CLI flag has a sensible default and is documented via <code>--help</code> ; the annotated output video gives immediate visual feedback.
Maintainability	The codebase is split into four single-responsibility modules (models, detector, tracker, reporter) connected via interfaces.
Scalability	Because tracking cost depends on the number of on-screen vehicles rather than total video length, the tool scales well to long videos.
Error Handling	Missing files, unreadable videos, and out-of-range arguments raise clear, user-facing errors before any processing begins.

5. System Architecture

The system follows a linear pipeline architecture: each module consumes the previous module's output and has no knowledge of any module beyond its immediate neighbor. This keeps each piece independently testable and replaceable (e.g. a different detector model can be swapped in without touching the tracker or reporter).

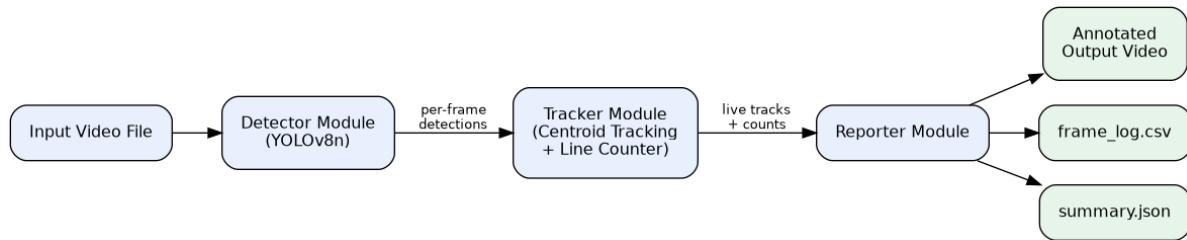


Figure 1 — System Architecture: data flow from input video to output reports.

6. Design Diagrams

6.1 Process / Workflow Diagram

Shows the full control flow of a single run, from CLI argument parsing through per-frame processing to final report writing.

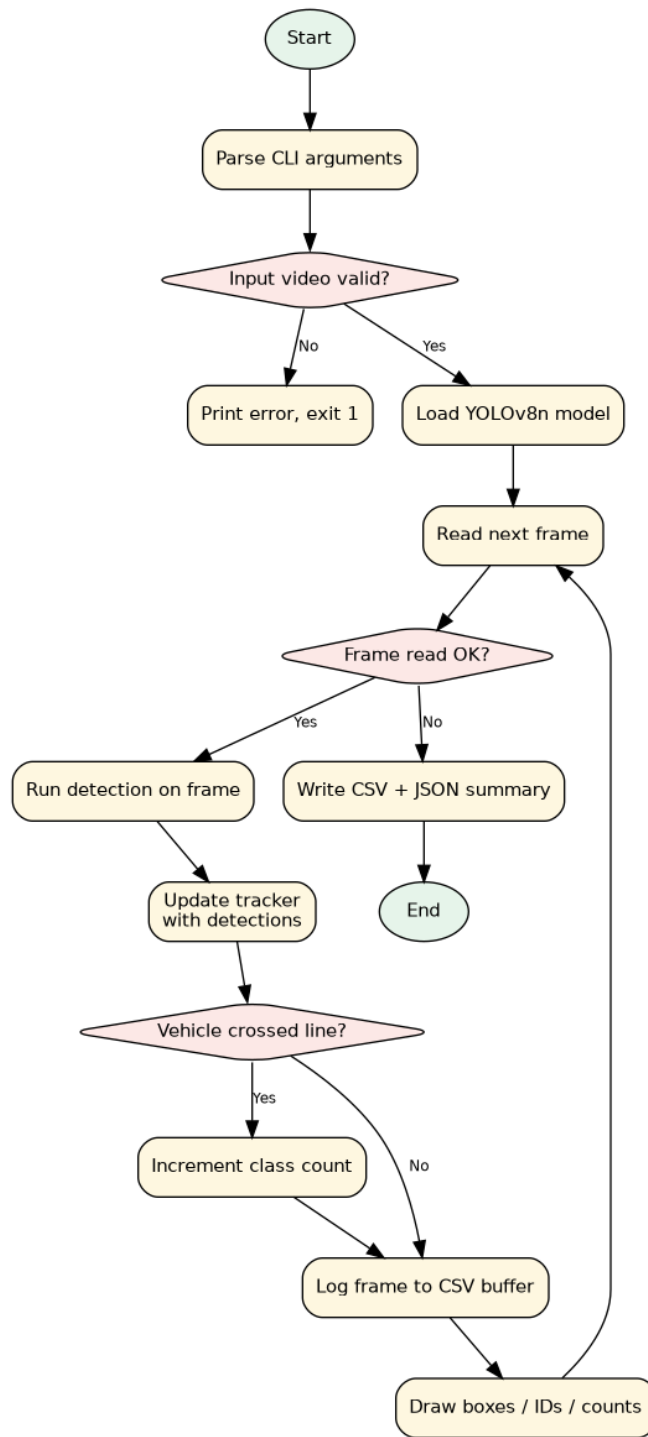


Figure 2 — Workflow of a full program run.

6.2 Use Case Diagram

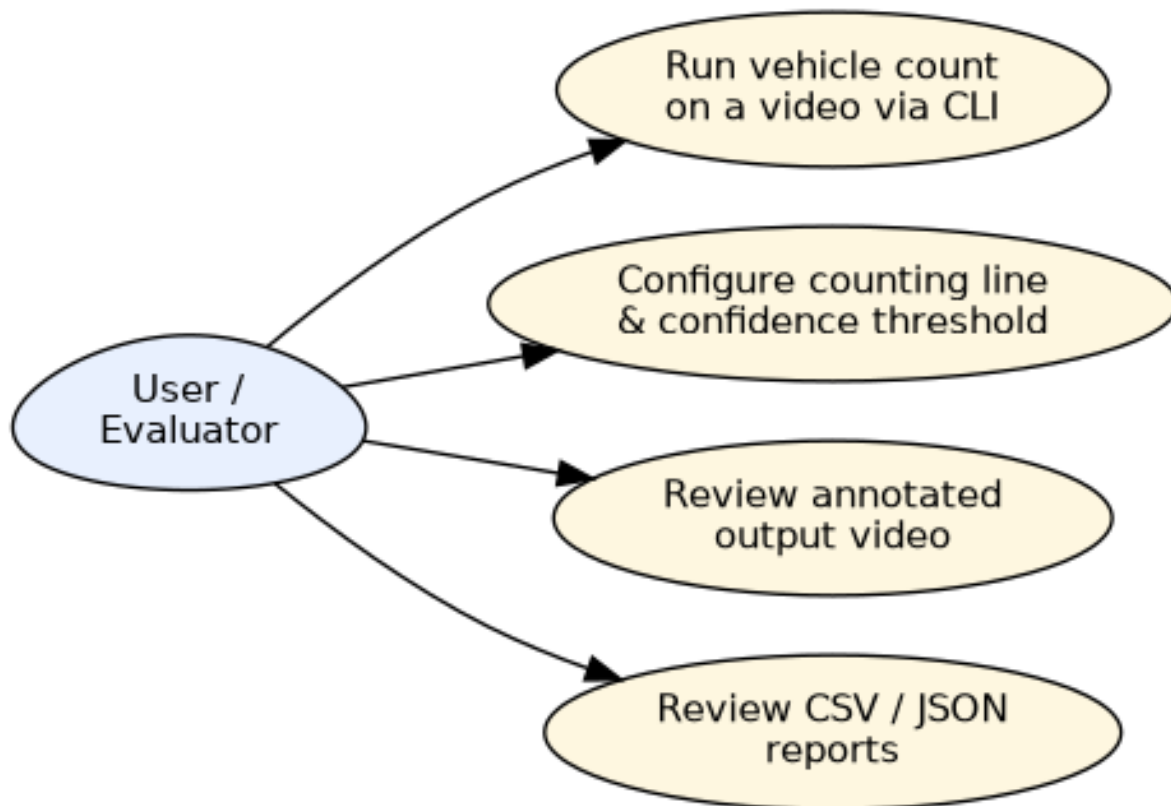


Figure 3 — Use cases available to the user/evaluator via the CLI.

6.3 Class / Component Diagram

Reflects the actual module structure of the codebase (src/models.py, detector.py, tracker.py, reporter.py).

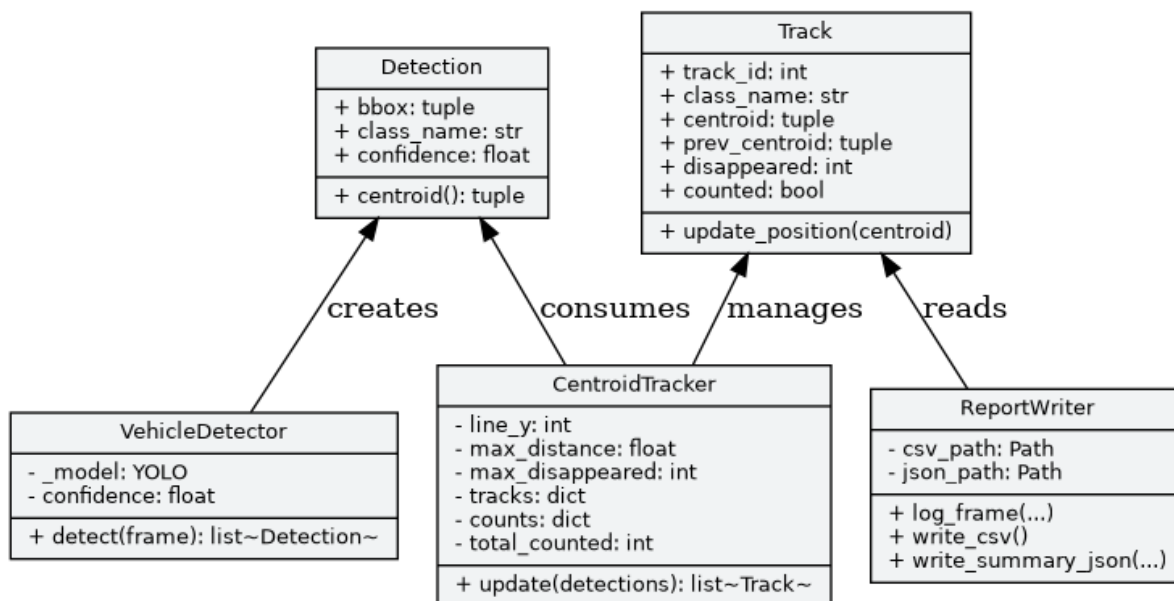


Figure 4 — Class relationships across the four core modules.

6.4 Sequence Diagram

Illustrates the calls made once per video frame during processing.

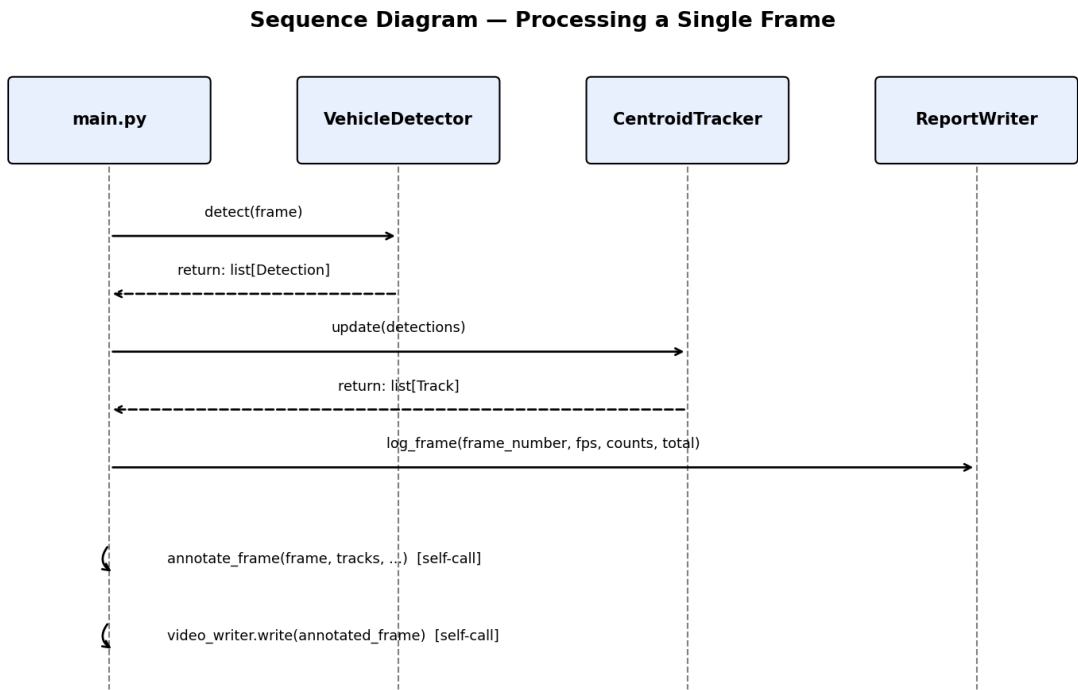


Figure 5 — Sequence of calls for processing a single frame.

6.5 Database / Storage Design

This project intentionally uses file-based storage (CSV and JSON) rather than a database, since a single video-processing run produces a self-contained report with no need for persistent querying or multi-user access. No ER diagram applies. The CSV schema is: `frame, timestamp_sec, total_count, <count per vehicle class>`, one row per processed frame.

7. Design Decisions & Rationale

- **Pretrained detector, custom tracker:** Training a detector from scratch would need a labeled vehicle dataset and GPU time disproportionate to this project's scope. YOLOv8n is pretrained on COCO and already recognizes car/truck/bus/motorcycle well, so effort was spent on the tracking and counting algorithm instead — the part that is genuinely specific to this problem and not available off-the-shelf.
- **Greedy centroid tracking over a heavier tracker (e.g. DeepSORT):** Centroid tracking is simple to reason about, fast enough for CPU-only use, easy to unit test with synthetic data, and sufficiently accurate for a single fixed-camera counting line, which is this project's actual requirement.
- **Line-crossing counting instead of counting all detections:** Counting raw detections per frame would wildly over-count (the same vehicle appears in dozens of frames). Counting only on line-crossing, combined with a per-track 'counted' flag, guarantees each vehicle is counted exactly once regardless of how long it stays on screen.
- **Separate models.py module:** The Detection data class was pulled out of detector.py into its own dependency-free module specifically so the tracker (and its unit tests) never need the ultralytics/YOLO dependency — a deliberate decoupling decision made after the initial design, to keep the core algorithm testable in isolation.
- **CLI-only, no GUI:** Matches the submission requirement that the project be fully executable from the command line, and keeps the tool scriptable for batch processing.

8. Implementation Details

8.1 Detection Module (src/detector.py)

Wraps ultralytics' YOLO class. On each call to `detect(frame)`, it runs inference at a configurable confidence threshold and filters results down to the four COCO class IDs that correspond to road vehicles (2=car, 3=motorcycle, 5=bus, 7=truck), returning a list of lightweight `Detection` objects.

8.2 Tracking & Counting Module (src/tracker.py)

Implements `CentroidTracker`, a greedy nearest-centroid matching algorithm: on every frame it computes Euclidean distances between all existing track centroids and all new detection centroids, matches the closest pairs first (below a max distance threshold), ages out unmatched tracks after `N` consecutive misses, and registers unmatched detections as brand-new tracks. A separate line-crossing check compares each track's previous and current y-coordinate against the counting line, incrementing the count exactly once per track via a boolean 'counted' flag.

8.3 Reporting Module (src/reporter.py)

`ReportWriter` buffers one row per frame (frame number, timestamp, running counts) and writes it to CSV at the end of the run, plus a final JSON summary with totals by class and run metadata. A separate `annotate_frame()` function draws the counting line, live track markers with ID/class labels, and a running count overlay directly onto each video frame using OpenCV drawing primitives.

8.4 CLI Entry Point (src/main.py)

Uses `argparse` to expose all configuration as flags, validates the input path and argument ranges before doing any expensive work, then drives the main per-frame loop: read frame → detect (respecting `--skip-frames`) → update tracker → log to report → annotate and write to output video.

9. Screenshots / Results

The full pipeline (tracker + reporter + annotation) was validated end-to-end using a synthetic test video generated with OpenCV, simulating two vehicles (one 'car', one 'truck') moving through the frame and crossing the counting line. This validates the tracking, counting-once, and reporting logic independently of the YOLO model download step, which requires network access on first run. The frames below are taken directly from the annotated output video produced by the tool.

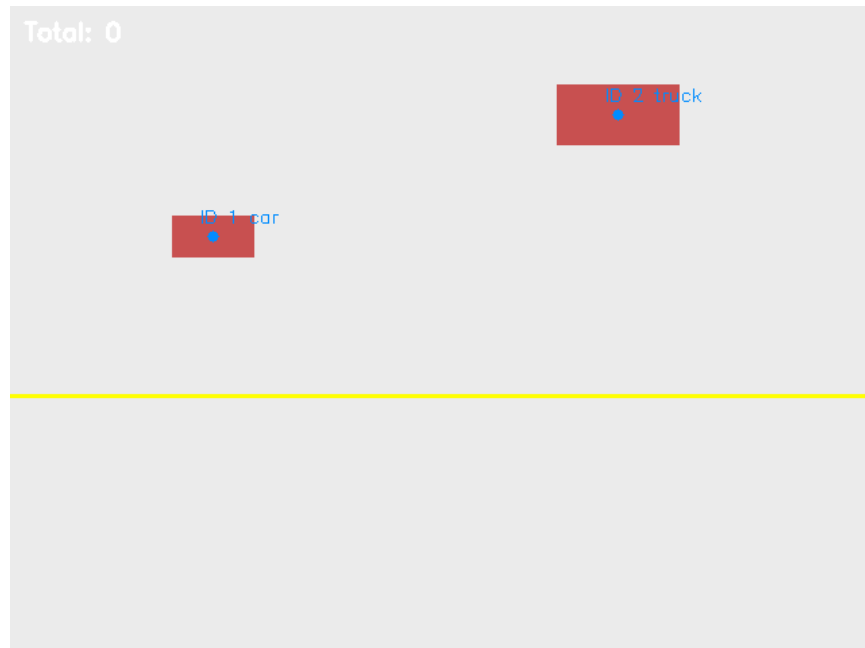


Figure 6 — Frame 30: two vehicles approaching the counting line, not yet counted.

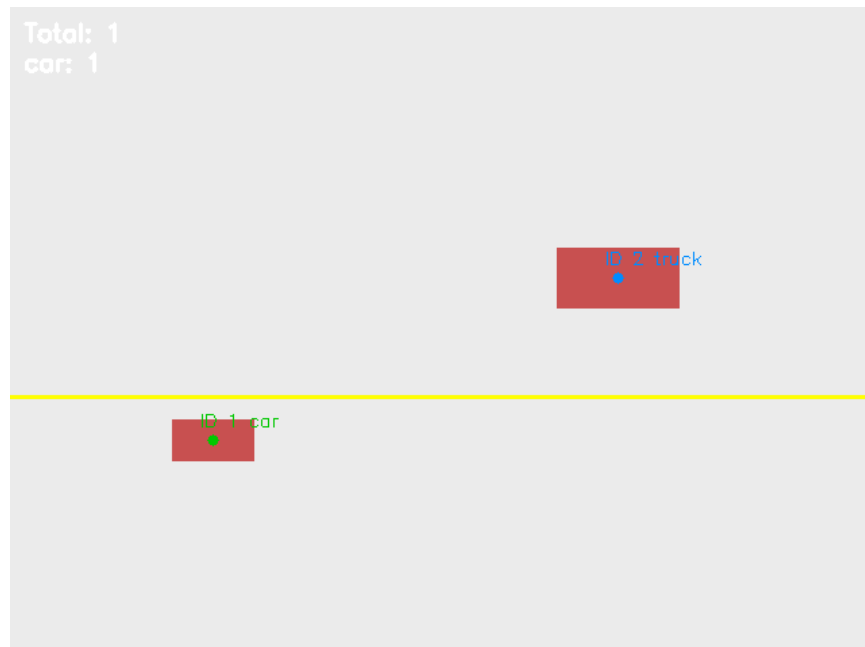


Figure 7 — Frame 60: the car has crossed the line and been counted once (Total: 1).

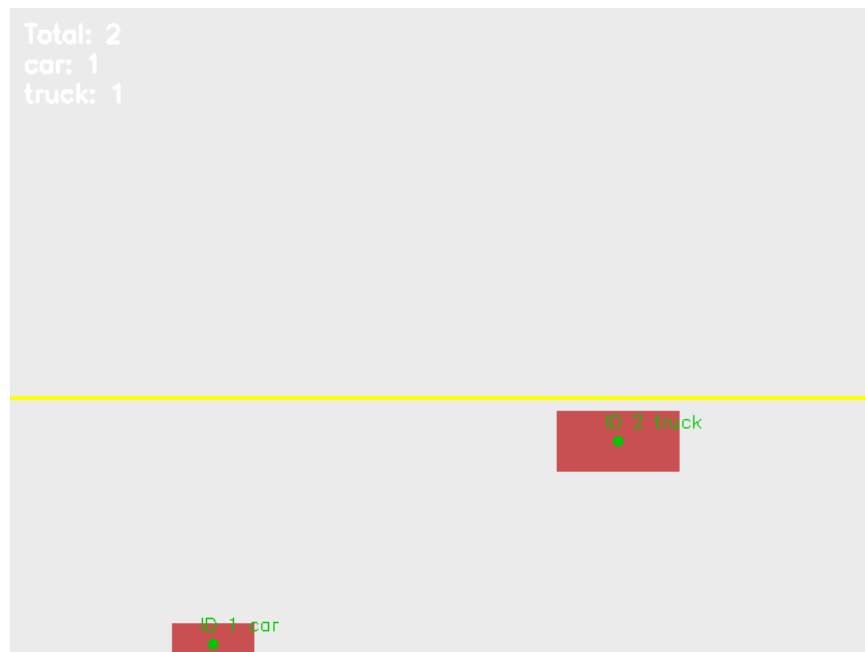


Figure 8 — Frame 90: both vehicles have crossed and are counted correctly (Total: 2, car: 1, truck: 1).

Sample summary.json output

```
{
  "video_source": "synthetic_demo",
  "frames_processed": 90,
  "video_duration_sec": 4.5,
  "total_vehicles_counted": 2,
  "counts_by_class": { "car": 1, "truck": 1 }
}
```

On a real traffic video, the same pipeline runs unchanged — only the detections come from YOLOv8n instead of synthetic rectangles. Users can generate this by running `python src/main.py --input <video>` as described in the README.

10. Testing Approach

Testing focuses on the tracker and reporter — the two modules with genuine algorithmic logic — using synthetic data so tests run instantly without a video file or the ML model.

- **Tracker tests (6 cases):** new-track registration, ID persistence across frames for a slowly moving object, stale-track removal after `max_disappeared` misses, single-count-only line-crossing (including no double-count after further movement), per-class count separation, and correct handling of a detection too far away to match an existing track.
- **Reporter tests (2 cases):** CSV rows match logged frame data exactly; JSON summary contains the correct totals and metadata fields.
- **End-to-end validation:** a synthetic video run (Section 9) exercises the full pipeline — detection-shaped input, tracking, counting, CSV/JSON writing, and video annotation — together in one run, confirming the modules integrate correctly.

All 8 unit tests pass. Run with: `python -m pytest tests/ -v`

11. Challenges Faced

- **Avoiding double-counting:** An early version counted a vehicle every frame its centroid was past the line, wildly over-counting. Solved by adding a one-time 'counted' flag per track, so crossing is a discrete event, not a per-frame state.
- **Matching detections to tracks stably:** Nearest-neighbor matching alone could occasionally swap IDs when two vehicles passed close together. A max-distance cap on valid matches (so a detection can only match a track within a plausible movement radius) reduced this significantly.
- **Testing without the ML model:** The tracker and reporter needed to be testable without requiring ultralytics/YOLO to be installed (a heavy, network-dependent dependency). This drove the decision to extract the Detection data class into its own dependency-free `models.py` module.

12. Learnings & Key Takeaways

- Detection and tracking are genuinely separate problems — a strong detector alone does not give you identity or counts across time; that requires its own algorithm.
- Designing for testability up front (isolating the tracker from the ML dependency) made it possible to verify correctness quickly and repeatedly, rather than only being able to check results by eye on a video.
- Small algorithmic decisions (e.g. exactly when to mark something 'counted') have an outsized effect on real-world correctness compared to the detector's accuracy.

13. Future Enhancements

- Multi-line counting to support directional flow (e.g. entering vs. exiting a junction).
- Speed estimation using frame-to-frame pixel displacement and a camera calibration step.
- Swap the greedy centroid tracker for an IoU- or appearance-based tracker (e.g. DeepSORT) for more robust identity preservation in dense traffic.
- A lightweight web dashboard to visualize the CSV/JSON output over time (optional, since the core tool is intentionally CLI-first).

14. References

- Ultralytics YOLOv8 documentation — <https://docs.ultralytics.com/>

- OpenCV Python documentation — <https://docs.opencv.org/>
- COCO dataset class list — <https://cocodataset.org/>
- Centroid tracking algorithm concept — a common, well-documented approach for lightweight multi-object tracking without a deep re-identification model.